AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 7

Design Problem: An Enterprise Knowledge Assistant

Fifty thousand employees, four million documents, and permissions that differ for every team. An engineer asks about a confidential reorganization, and the answer must depend on who's asking. This lesson puts the module together.

Module	AI05 of 13
Lesson	7 of 7, design problem
Prerequisites	AI05 L1–L6, AI04 L4–L5, AI03 L5
Exam relevance	Out of scope as a design. Provenance and error handling support TS 5.3 and TS 5.6
Reading time	~60 minutes

Contents

1. Design an Enterprise Knowledge Assistant
2. Module Revision

Lesson 7 of 7 · RAG

Design an Enterprise Knowledge Assistant

Key idea

At this scale, the hard parts aren't vector search or queries per second. They're permissions, freshness and knowing whether answers are right. Peak load here is about 8 questions a second. **The design has to fail closed on permissions, abstain when evidence is weak, and measure both halves of the pipeline by segment.**

> *Before: all of AI05. Teaches: putting the pieces together, using the ten-part design template. Used by: AI08 L5 (Java build), AI09 (threats), AI10 (operations).*

This is a recommended reference design, not a specific company's system. All sizes and rates are stated assumptions, and costs are illustrative.

1. Scope

Build: an assistant that answers questions from company knowledge, with citations, respecting each employee's access.

In scope	Out of scope for version 1
Wikis, drives, policies, ticket archives	Taking actions in other systems
Answers with citations	Whole-corpus analytics
Per-user permissions	Multi-step research agents
Follow-up questions	Images inside documents
Saying "I don't know"	Writing back to sources

Why this scope? LinkedIn's write-up shows that the last stretch of quality consumes the schedule. A narrow first version reaches good quality sooner, and the excluded items can be routes added later.

2. Requirements

Type	Requirement	Target (engineering choice)
Functional	Cited answers from permitted documents	Every claim cited
Functional	Admits when it doesn't know	Evaluated as a segment
Security	**Never reveals a document the user can't access**	Zero tolerance, tested
Freshness	Edits appear quickly	95% within 15 minutes
Freshness	Revocations apply quickly	Re-checked at read time for sensitive sources
Latency	First token	p95 under 3 seconds
Availability	Assistant available	99.9%, and failures must be visible
Quality	Recall@20, faithfulness	Gates by segment, chosen from a measured baseline

3. Estimation

These are illustrative assumptions, recomputed in `scripts/verify_ai05.py`.

Quantity	Calculation	Result
Daily active users	50,000 × 40%	20,000
Questions per day	20,000 × 4	80,000
Average rate	80,000 ÷ 8-hour workday	2.8 per second
Peak rate	Average × 3	**8.3 per second**
Chunks	4M documents × 12	**48M**
Vectors, float32, 1024 dimensions	48M × 4,096 bytes	197 GB
Vectors, int8	48M × 1,024 bytes	49 GB
HNSW graph (as in AI04 L5)	48M × 264 bytes	13 GB
int8 and graph		**62 GB**
Chunk text	48M × 1,600 bytes	77 GB

Per question, on current list prices:

Step	Tokens	Cost
Rewrite, Haiku 5.5 ($0.10 / $0.50)	600 in, 60 out	$0.00009
System prompt, Sonnet 5.5 cache read	2,000	$0.00020
8 chunks, Sonnet 5.5 ($2 per million)	3,200	$0.00640
Answer, Sonnet 5.5 ($10 per million)	450 out	$0.00450
Total		**$0.0112**

Monthly and one-time	Calculation	Result
Questions per month	80,000 × 22 working days	1.76M
Model cost	1.76M × $0.0112	**about $19,700**
Per employee	÷ 50,000	**about $0.39**
Contextualize once	19.2B tokens × $1.02 per million	about $19,600
Monthly, if 5% of content changes	0.96B tokens × $1.02 per million	about $980

The $1.02 is Anthropic's 2024 figure, for Haiku 3. Treat it as a scale estimate, and recompute with current pricing and your document sizes. Infrastructure, embeddings and reranking aren't included. Prompt-cache writes barely matter here, at most about $0.48 a day, since a busy system keeps the cache warm.

What the numbers say

> *Key idea*
>
> **8 questions a second is small.** Serving capacity isn't the problem, and the index fits on few machines using int8. Model cost is about 39 cents per employee per month. **So spending engineering effort on exotic scaling would be wasted, and it should go to permissions, ingestion and evaluation.**

The rate limit is worth checking, though. At peak, it's about 1.6M uncached input tokens and about 224,000 output tokens per minute, so confirm your tier.

4. API

```
POST /v1/ask
Authorization: Bearer <employee token>

{ "conversation_id": "c_81", "question": "What about contractors?" }
```

Streamed response, using server-sent events:

```
event: status    {"stage": "searching"}
event: delta     {"text": "Contractors aren't eligible for "}
event: citation  {"document_id": "hr-114", "version": 7, "title": "Leave Policy",
                  "cited_text": "...", "url": "..."}
event: done      {"outcome": "ANSWER", "trace_id": "t_55", "interpreted_query":
                  "Parental leave policy for contractors"}
```

Decision	Why
Identity comes from the token, never the body	A user can't ask as someone else
`outcome`: `ANSWER`, `NOT_FOUND`, `UNAVAILABLE`	"No information" and "it's broken" are different, and clients should show them differently
Returns `interpreted_query`	Users can see a bad rewrite
Returns `trace_id`	Feedback links to the full trace

5. Data model

```
-- Durable source of truth for search indexes (recommendation, not run).
CREATE TABLE documents (
  document_id     text PRIMARY KEY,
  source          text NOT NULL,
  active_version  int,              -- search reads only this version
  allowed_groups  text[] NOT NULL,
  deleted_at      timestamptz
);
CREATE TABLE chunks (
  document_id    text,
  version        int,
  chunk_no       int,
  heading_path   text,
  body           text NOT NULL,
  context        text,              -- Contextual Retrieval prefix
  content_hash   text NOT NULL,
  embedding_model text NOT NULL,
  PRIMARY KEY (document_id, version, chunk_no)
);
CREATE TABLE traces (
  trace_id text PRIMARY KEY, employee_hash text, question text, rewrite text,
  chunk_ids text[], outcome text, prompt_version text, created_at timestamptz
);
```

The search index copies `allowed_groups`, version and date to each chunk, so that filters run **during** search (AI04 L3).

6. High-level design

V1: stuff or simple search. 4M documents and per-user permissions rule out a stuffed prompt. A plain vector search over everything looks like this:

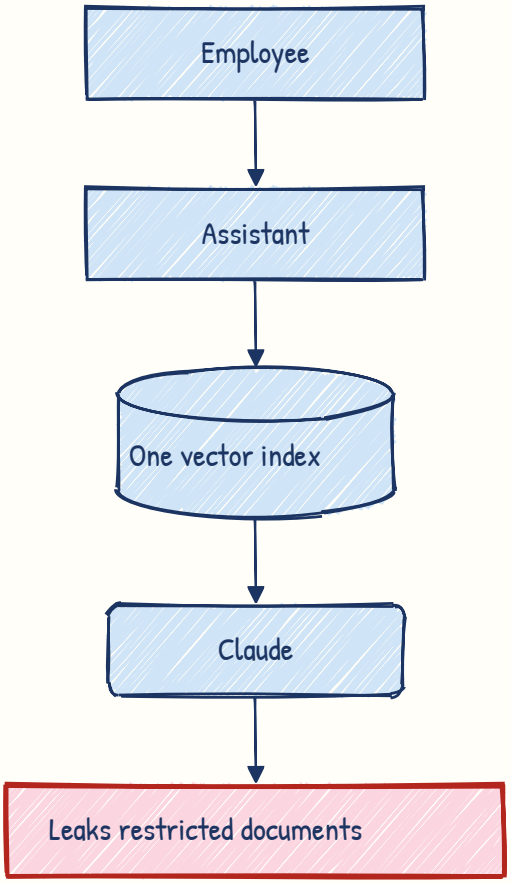


Figure 1. Baseline: a global index searched with no permission logic. Any employee can reach any document through the model's answer.

V2: a filter added, and hybrid search. Permissions apply during search, and keyword search handles exact terms. New problem: **the group list can be stale, edits appear partially, and nobody knows the quality.**

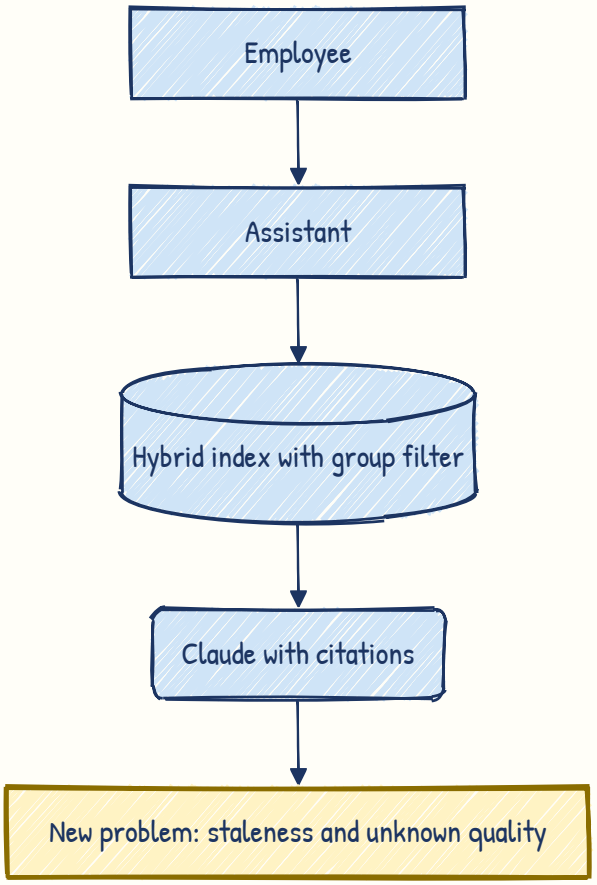


Figure 2. Improved: permission-filtered hybrid search, which still lacks fresh revocations, atomic updates and measurement.

V3: production.

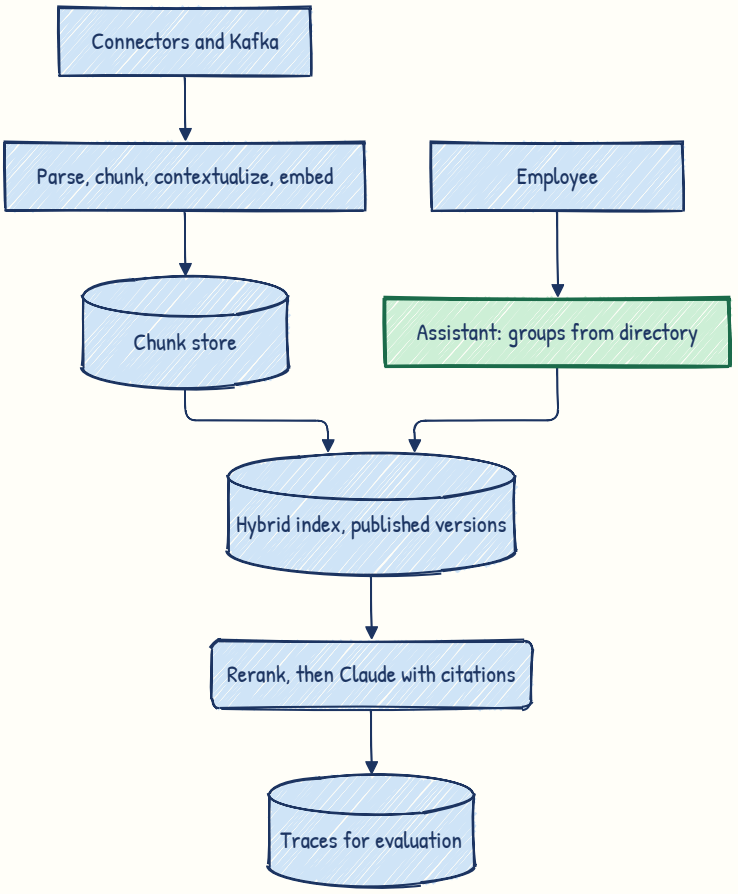


Figure 3. Production. Ingestion and the query path meet only at the index. Ingestion publishes document versions atomically, and the query path filters by current groups, reranks, cites and logs.

Component	Why it exists	How it communicates
Connectors	Pull changes and permissions from each source	Webhooks or polling, into Kafka
Kafka	Decouples bursts, and keeps per-document order	Keyed by document ID
Ingestion workers	Slow, retryable work	Consumers, calling the model and embedding APIs
Chunk store	Lets you rebuild indexes (AI04 L4)	SQL
Hybrid index	Fast filtered search	Search API
Directory	Current groups for a user	Cached call with a short TTL
Assistant service	Orchestrates, and owns decisions	Stateless HTTP, streams to client
Reranker	Precision at the top	API
Claude	Cited generation	Messages API, streaming
Traces	Evaluation and debugging	Async writes

Request flow

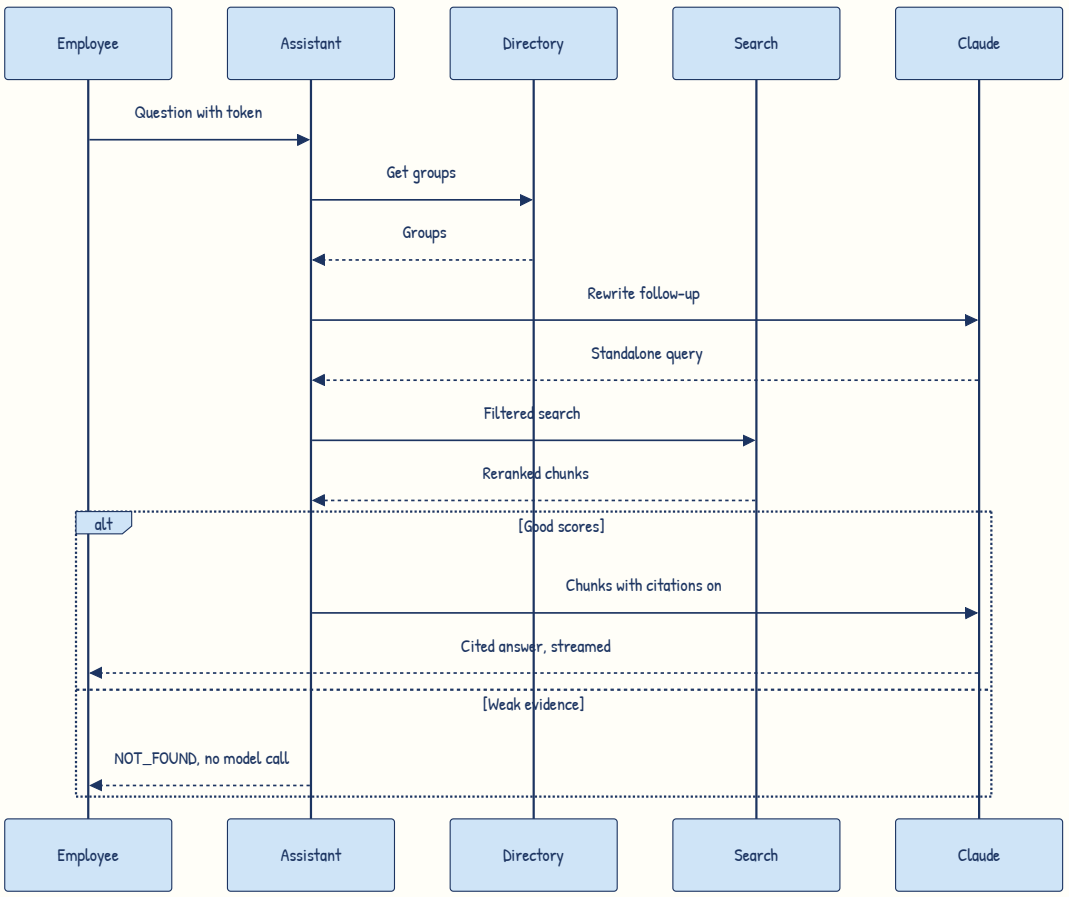


Figure 4. Request flow. Permissions are looked up before search, and when evidence is weak, the service abstains without paying for a model call.

7. Deep dives

Deep dive 1: permissions, the part that can't fail

Rule: the filter applies during search, and failure means no answer. Here's real output from `KnowledgeAssistant.java`, which uses fake services to test the decisions around the model call:

```
// Unknown permissions mean no search at all, never an unfiltered one.
Optional<Set<String>> groups = directory.groupsOf(employee);
if (groups.isEmpty()) {
    return new Reply("UNAVAILABLE", "permissions could not be checked", false);
}
List<Chunk> usable = search.find(question, groups.get()).stream()
        .filter(c -> c.rerankScore() >= MIN_RELEVANCE)
        .toList();
if (usable.isEmpty()) {
    return new Reply("NOT_FOUND", "no accessible document covers this", false);
}
```

```
asha            Reply[kind=ANSWER, detail=2 chunks sent to the model, modelCalled=true]
ravi            Reply[kind=NOT_FOUND, detail=no accessible document covers this, modelCalled=false]
directory-down  Reply[kind=UNAVAILABLE, detail=permissions could not be checked, modelCalled=false]
```

The threshold of 0.40 is a placeholder, and you'd tune it on your eval, since scores differ by reranker.

Threat	Mitigation
Group list is stale	Short TTL, plus a re-check at read time for sensitive sources
Source permissions change, index lags	Permission events get priority, and a monitor tracks lag
A shared cache returns someone else's answer	**Key caches by permission set, or don't share** (AI04 L5)
`NOT_FOUND` reveals existence	The same message for "absent" and "not permitted"
Prompt injection in a document	Treat as untrusted (AI09 L2). The assistant has no write tools, which limits the damage
Group explosion in filters	Use the search engine's filter strategies (AI04 L3)

Test this as a segment: questions whose answer exists only in a document that this user can't see. The correct result is `NOT_FOUND`.

Deep dive 2: freshness

Versioned publishing, as in AI05 L2. Each edit is written fully, then its version is published.

Event	Target	How
Edit	15 minutes	Event, then worker, then publish
Delete	Minutes	Tombstone first, and clean up after
Revocation	Fast, or re-check	Priority topic
Connector misses something	Daily	Reconcile against source lists

Show the document's date in each citation, so users can judge staleness themselves.

Deep dive 3: the retrieval stack

Step	Choice	Why
Chunking	Structure-aware, about 400 tokens	AI05 L2
Context	Contextual prefix, if the eval justifies it	Reduced retrieval failures in Anthropic's tests (AI05 L3)
Search	Embeddings and BM25, top 150	AI04 L3
Fusion	RRF	AI04 L3
Rerank	To 8 at first, then tune	Anthropic's tests used 20. Going to 20 here would raise context cost from $0.0064 to $0.016 per question, so test whether that pays
Store	A search engine with both indexes, such as OpenSearch	One filtered query, and keyword search built in (AI04 L4)

Deep dive 4: knowing that it works

Gates by segment (AI05 L6): policy lookup, exact terms, multi-document, follow-ups, no-answer, **no-access**, stale policy. The no-access segment has a zero-failure rule, and **any single failure blocks release.**

8. Scaling

Scale what's actually limiting. Here's a sensible order:

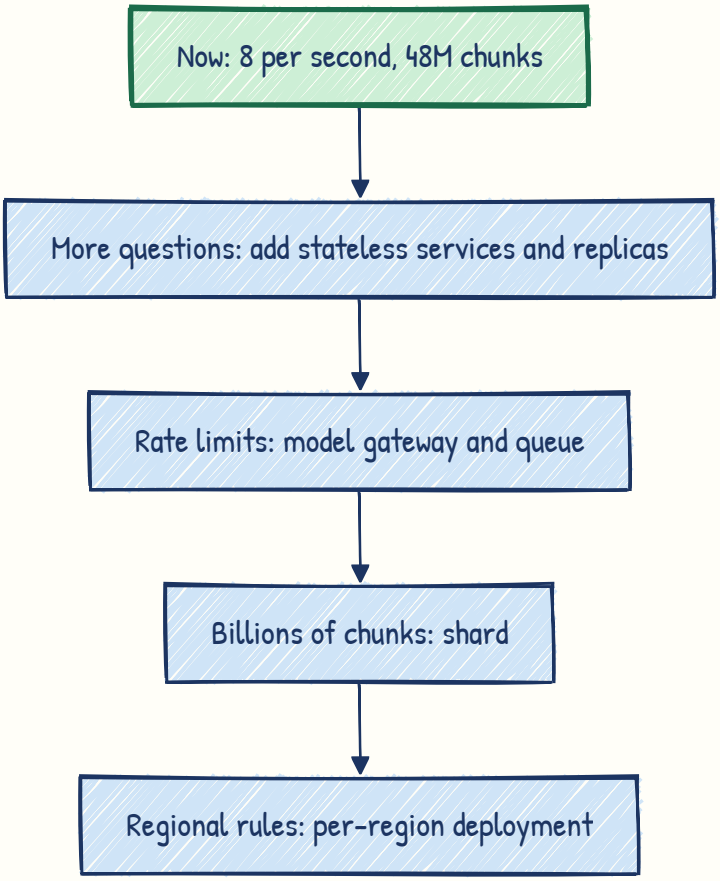


Figure 5. Scaling stages, in the order they typically become necessary. Each step is taken only when a measured limit is reached.

Limit	Signal	Response
Search latency	p95 rising	Replicas, or int8 and two-stage search
Model rate limits	429s at peak	Gateway, higher tier, backoff (AI03 L1)
Ingestion backlog	Consumer lag	More workers, and prioritize permission events
Costs	Budget	Model routing, a cache keyed by permission set

A semantic cache? At a 20% hit rate, it might save about $3,900 a month in answer generation. But a shared answer can leak across permissions and go stale. So I wouldn't start with one here. A cache per permission set is safer, though with a lower hit rate.

Failure handling

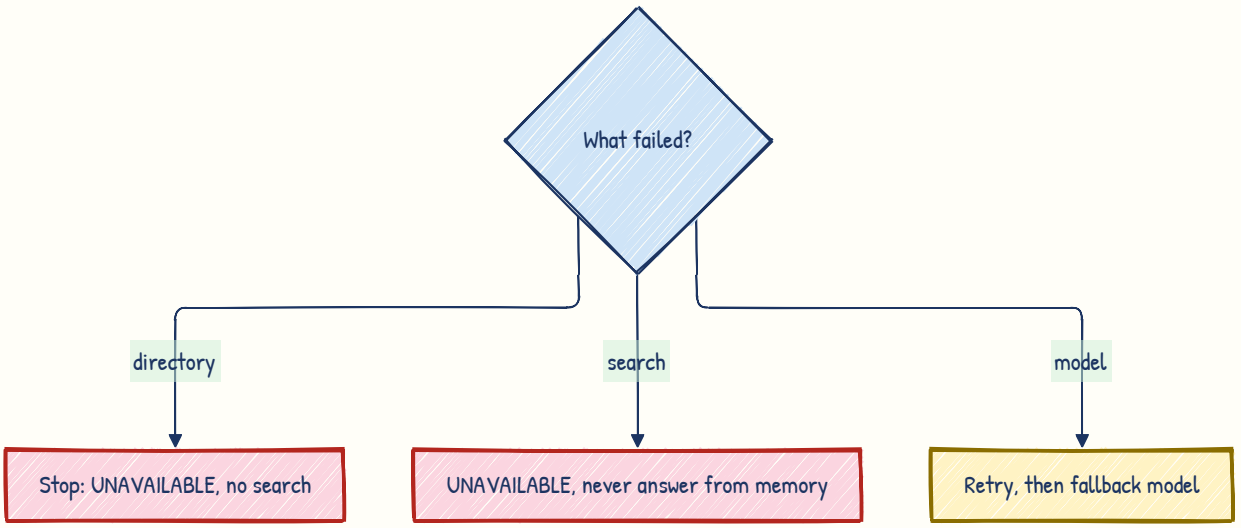


Figure 6. Failure routing. Permission failures stop the request, retrieval failures are reported as unavailable, and model failures can fall back.

Failure	Detection	Containment
Directory down	Errors	**Fail closed**
Search down	Errors	`UNAVAILABLE`. Never let the model answer from its own knowledge
Reranker down	Errors	Use fused results, and record that it degraded
Claude overloaded	529, or 429 with retry-after	Backoff, then a fallback model
Ingestion stuck	Consumer lag	Serve last published versions, show dates, and alert
Bad parser release	Sampled parse check	Roll back, and re-ingest affected documents
Bad prompt release	Canary by segment	Roll back the version

9. Trade-offs

Decision	Chosen	Alternative	Why
Permissions	Filter in search, with a re-check for sensitive sources	Filter after search	The model must never read restricted text. Post-filtering can also return too few results
Failure mode	Fail closed	Serve anyway	Leaks outweigh downtime here
Model	Sonnet 5.5, Haiku for rewriting	Opus for all	Evaluate before paying more
Cache	No shared answer cache at first	Semantic cache	Permission risk
Scope	Single-shot retrieval	Agent	Cheaper, and simpler to evaluate. Add routes later
Index	One hybrid search engine	Separate vector and keyword engines	Simpler filter logic
Weak evidence	Abstain without a model call	Let the model decide	Saves money. Tune it, or users get too many refusals

Where this would be overkill: a single team with a few hundred documents and the same access for everyone. Use a cached prompt, or a pgvector table.

10. Follow-up questions

Q Follow-up 1. Security asks: "Prove that no one can see documents they shouldn't." What do you offer?

A A test suite of no-access questions by group, gated in CI, with a zero-failure rule. An audit log of chunk IDs and the groups used for each answer. A daily job comparing a sample of indexed permissions with the source. I'd be honest that this gives strong evidence rather than proof, and that lag between a source revocation and the index remains, which is why sensitive sources are re-checked at read time.

Q Follow-up 2. Users say answers are often right but outdated. What do you check, in order?

A Ingestion lag for that source. Whether superseded documents remain published, or duplicates outrank the current one. Whether the ranking uses dates. Add a stale-policy segment to the evals, display dates, and use effective-date filters where documents have them.

Q Follow-up 3. Leadership wants "What are the top employee complaints this year?" Does this design handle that?

A No, top-k can't see the whole corpus. Also, any aggregate over HR documents raises permission questions. I'd build it as a separate batch job with its own access controls, or a graph or clustering route, rather than stretching this path (AI05 L5).

Q Follow-up 4. The company wants to add actions, such as "file my leave request." What changes?

A This changes the risk class. A prompt injection in a retrieved document could now trigger an action. So actions need the user's own permissions, explicit confirmation, deterministic checks and idempotency, with reads and writes separated (AI06 L8, AI09 L2).

Q Follow-up 5. Finance says that $19,700 a month is too high. Where do you look first?

A Check the logs, and find out which is largest. Input context is $0.0064 of the $0.0112, so test fewer or shorter chunks against the eval. Try Haiku on simple question types using routing, if evals pass. Consider a cache keyed by permission set for very common questions. And compare with the time it saves. At about $0.39 per employee per month, it may already be good value.

Module Revision

Revision - AI05

→ Check first: a small, uniformly permitted corpus can go in a cached prompt. Live data needs tools, and aggregates need SQL.

→ Most RAG quality is decided at ingestion. Read your parsed chunks.

→ Chunk by structure, and keep heading paths. Measure with fact integrity.

→ Publish document versions atomically, to prevent orphan chunks.

→ Contextual Retrieval took the failure rate from 5.7% to 1.9% in Anthropic's tests. Measure on yours.

→ Rewrite follow-up questions, and log both forms.

→ Citations guarantee valid pointers, not support.

→ Provenance must be structural. Annotate conflicts, and require dates.

→ Report coverage gaps, since silence may be an outage.

→ Each RAG variant answers a kind of question that top-k can't.

→ SQL safety lives in database permissions.

→ Score retrieval and generation separately, by segment.

→ Include no-answer and no-access cases.

→ Calibrate judges item by item.

→ At enterprise scale, permissions, freshness and evaluation are harder than throughput.

→ Fail closed on permissions, and never answer from memory when search is down.

Self-assessment

#	Without notes, can you answer?
1	What are the three checks before building RAG?
2	Why can a large cached prefix cost less than a small uncached one?
3	What does RAG not fix?
4	What did LinkedIn find about the last stretch of quality?
5	What's a fact-integrity test?
6	Why did structure-aware chunking beat heavy overlap in the lab?
7	How do orphan chunks arise, and how do you prevent them?
8	What is parent-child retrieval?
9	Which metadata fields matter, and why?
10	What does Contextual Retrieval add, and when?
11	Give its published figures, and their limits.
12	Why rewrite a follow-up question?
13	What's HyDE's idea, and its cost?
14	State the order for assembling context.
15	What do Claude's citations guarantee, and what don't they?
16	What can't citations combine with? What's the workaround?
17	How do you keep attribution through summarization?
18	How do you handle sources that disagree in the same year, and in different years?
19	What goes in a coverage gaps section, and why?
20	Name the nine retrieval shapes, and one reason for each.
21	When is a batch job better than GraphRAG?

#	Without notes, can you answer?
22	What did Uber find, and what did it leave unsolved?
23	Where does SQL safety really come from?
24	Why can't you directly apply Self-RAG to a hosted model?
25	Why do faithful and correct differ?
26	Which segments must a golden set include?
27	What's the margin of error at 20 cases?
28	How do you calibrate a judge? What trap do totals hide?
29	Why is 8 per second the wrong thing to worry about here?
30	Describe how the design behaves when the directory is down, and when search is down.

Where this module is used

Idea	Where it returns
Tool-based retrieval, agentic routing	AI06 L3, AI06 L6
Provenance and coverage	AI06 L6, AI06 L10, AI13 L6
RAG service in Java	AI08 L5
Injection through documents, access control	AI09 L2, AI09 L4
Evaluation pipelines	AI10 L2–L3

Working space